

Free Form Testing Agent (FFTA)

Phase 1 Progress Report

Building the Foundation of an Autonomous Software Testing Agent

Version: Phase 1 - Milestone 6

Date: June 2026

1. Project Vision

The goal of the **Free Form Testing Agent (FFTA)** is to build a fully autonomous software testing agent capable of:

- Launching applications
- Discovering UI elements
- Understanding possible actions
- Exploring applications without predefined test cases
- Building state graphs
- Learning workflows
- Reproducing bugs
- Generating test cases
- Eventually using AI models to guide exploration and reasoning

The long-term objective is to create a system that treats software applications as an explorable state space.

2. Core Philosophy

Traditional automation:

```
graph TD; A[Test Case] --> B[Execute]; B --> C[Verify];
```

FFTA philosophy:

```
Observe
  ↓
Understand
  ↓
Act
  ↓
Observe
  ↓
Learn
  ↓
Repeat
```

Or mathematically:

```
Software
    =
States
    +
Actions
    +
Transitions
```

3. Initial Architecture Decision

Instead of allowing the AI model to generate executable code:

```
LLM
  ↓
Generate Python Code
  ↓
Execute
```

we adopted a deterministic execution architecture:

```
LLM
  ↓
Generate Intent
  ↓
Framework Executor
```

↓
Perform Action

Example:

```
{  
  "action": "CLICK",  
  "target": "num7Button"  
}
```

Advantages:

- Deterministic
- Secure
- Explainable
- Easier debugging
- Reproducible execution
- Works with smaller local models

4. AI Model Selection

System configuration:

CPU:
Intel i7-14700HX

GPU:
RTX 4060 Laptop 8GB

RAM:
16GB DDR5

Selected local model:

deepseek-r1:8b

Future model architecture:

```
deepseek-r1
+
qwen2.5-coder
```

Responsibilities:

```
deepseek-r1:
  reasoning
  planning
  exploration

qwen:
  code generation
  tool usage
```

5. Project Structure

Created project architecture:

```
FreeFormTestingAgent/

agent/
core/
adapters/
storage/
prompts/
tests/
docs/
```

Submodules:

```
agent/
  explorer/
  executor/
  planner/
  memory/

core/
  models/
  graph/
  state/
```

```
adapters/  
  ui/  
  llm/  
  storage/
```

6. Core Domain Models

Implemented foundational models.

Action

Represents executable operations.

```
Action  
  
  action_id  
  action_type  
  target  
  value  
  description  
  timestamp
```

Supported:

- CLICK
- DOUBLE_CLICK
- TEXT_INPUT
- KEY_PRESS
- HOTKEY
- WAIT
- SCROLL

UIControl

Represents discovered UI elements.

```
UIControl
```

```
automation_id
name
control_type
class_name
depth
```

Purpose:

- Strong typing
- State comparison
- Action generation
- Filtering
- Serialization

ApplicationState

Represents a snapshot of an application.

ApplicationState

```
state_id
timestamp
window_title
controls
values
available_actions
metadata
state_hash
```

Transition

Represents:

```
State A
      +
Action
      =
State B
```

Structure:

Transition

source_state
target_state
action
success
duration

7. State Hashing

Implemented deterministic state hashing:

ApplicationState
↓
Serializable Object
↓
JSON
↓
SHA256

Purpose:

- Duplicate detection
- State comparison
- Workflow replay
- State graph creation

8. UI Automation Investigation

Initially attempted:

pywinauto

Result:

FAILED

Reason:

- Windows Calculator is a modern UWP application.
- pywinauto was unreliable.

Observed error:

```
ElementNotFoundError
```

9. Final UI Automation Strategy

Adopted:

```
uiautomation
```

Architecture:

```
Application Launch:  
    subprocess
```

```
UI Discovery:  
    uiautomation
```

```
Action Execution:  
    uiautomation
```

Advantages:

- Native Windows support
 - UWP support
 - Reliable UI tree access
 - Stable AutomationIds
-

10. WindowsUIAdapter

Implemented:

launch_application()

Launch executable

connect_window()

Connect to application window

get_controls()

Enumerate UI tree

capture_state()

Create ApplicationState

11. Recursive UI Discovery

Initial implementation:

```
Window
├── Child
└── Child
```

Problem:

Calculator buttons were hidden deeper.

Solution:

```
Window
  ↓
Child
  ↓
```

```
Child
  ↓
Button
```

Implemented recursive traversal.

12. First Successful Application Discovery

Successfully discovered:

```
Calculator Window
```

and all calculator controls:

```
num7Button
num8Button
plusButton
equalButton
...
```

This validated:

```
Application
  ↓
UI Tree
```

13. Action Discovery Engine

Implemented:

```
UIControl
  ↓
ActionDiscovery
  ↓
Action
```

Example:

Button:
 num7Button

 ↓

Action:
 Click Seven

Discovered actions:

Click Seven
Click Eight
Click Plus
Click Minus
Click Equals

14. Action Executor

Implemented:

Action
 ↓
ActionExecutor
 ↓
UI Automation
 ↓
Application

Successfully executed:

Click Seven

on Calculator.

This proved:

Observe
 ↓
Understand

Act



15. Dynamic State Extraction

Discovered that Calculator exposes:

```
CalculatorResults  
NormalOutput  
CalculatorExpression
```

Example:

Display is 0

After clicking:

7

Result:

Display is 7

This proved:

```
Application State  
    =  
UI Structure  
    +  
UI Values
```

16. State Transition Engine

Implemented:

```
State A
      +
Action
      =
State B
```

Successfully created:

```
Display=0
      |
Click(Seven)
      |
Display=7
```

Transition object:

```
f5240398
      |
Click Seven
      |
1453e59d
```

17. State Graph

Implemented:

```
StateGraph

    states
    transitions
```

Successfully stored:

```
===== STATES =====

Display=0

Display=7
```

```
===== TRANSITIONS =====
```

```
Display=0  
  |  
Click Seven  
  |  
Display=7
```

This is the first generated knowledge graph of the application.

18. Most Important Achievement

We successfully validated the central hypothesis of the entire project:

```
Software Application  
  ↓  
Can be represented as  
  ↓  
State Graph
```

Where:

```
Node:  
  ApplicationState  
  
Edge:  
  Transition  
  
Label:  
  Action
```

19. Exploration Memory

Implemented:

```
State  
  ↓  
Executed Actions
```

Example:

```
{
  state_1:
    {
      "num7Button",
      "plusButton"
    },
  state_2:
    {
      "num8Button"
    }
}
```

Purpose:

- Prevent loops
- Track coverage
- Enable autonomous exploration

20. Current System Architecture

```
Application
  ↓
WindowsUIAdapter
  ↓
ApplicationState
  ↓
ActionDiscovery
  ↓
ExplorationMemory
  ↓
ActionExecutor
  ↓
TransitionEngine
  ↓
StateGraph
```

21. Autonomous Agent Pipeline

Current pipeline:

```
Observe
  ↓
Capture State
  ↓
Discover Actions
  ↓
Filter Explored Actions
  ↓
Execute Action
  ↓
Observe New State
  ↓
Create Transition
  ↓
Update State Graph
  ↓
Repeat
```

22. What Has Been Proven

We have successfully proven:

```
Observe
  ✓

Understand
  ✓

Act
  ✓

Observe Again
  ✓

Learn
  ✓
```


This means FFTA is no longer an automation framework.

It has become the foundation of an autonomous software exploration agent.

23. Next Milestone

Phase 1 - Step 7:

Build:

```
AutonomousExplorer
```

Algorithm:

```
Capture State
  ↓
Find Unexplored Actions
  ↓
Select Action
  ↓
Execute
  ↓
Observe New State
  ↓
Create Transition
  ↓
Store Graph
  ↓
Repeat
```

24. Biggest Insight Discovered

The most important discovery of Phase 1:

```
Software
    =
States
    +
Actions
```

+
Transitions
+
Memory

This validates the entire Free Form Testing Agent architecture.